

01: Introduction

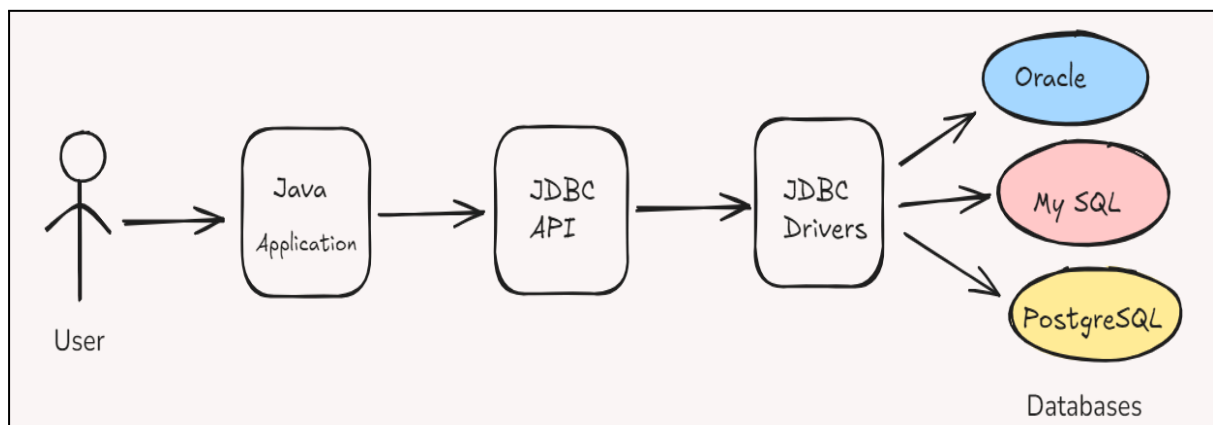
👉 What is JDBC?

Java Database Connectivity (JDBC) is a Java API that allows Java applications to interact with relational databases.

- It is a part of JavaSE (Java Standard Edition).
- The JDBC API uses JDBC drivers to connect with the database.
- The classes and interfaces of JDBC allow the application to send user requests to the specified database.

What is an API?

An API is a set of rules and protocols that enable communication between software applications, defining methods and data structures for developers to interact with the software component.



👉 Prerequisites for JDBC:

1. Java Programming Language
2. Database fundamentals
3. Relational Database Management System Language (SQL)

👉 Why to learn JDBC?

- **Frameworks Use JDBC:** Popular Java frameworks like Hibernate, Spring JDBC, and Spring Data JPA utilize JDBC. Understanding JDBC can help to optimize and troubleshoot these frameworks.

- **Troubleshooting and Optimization:** Knowing how JDBC works enables developers to debug issues effectively and optimize database interactions in Java applications.
- **Legacy Systems:** Some legacy projects still use raw JDBC, so knowledge of JDBC is essential for maintaining and updating such systems.

02: Need of JDBC

In the world of software development, data is at the core of most applications. Applications are built to process, manage, and store data, ensuring this data is stored and retrieved efficiently.

There are two types of storage:

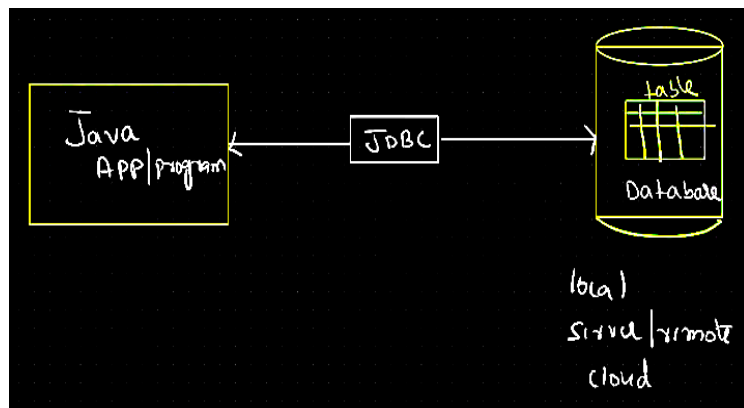
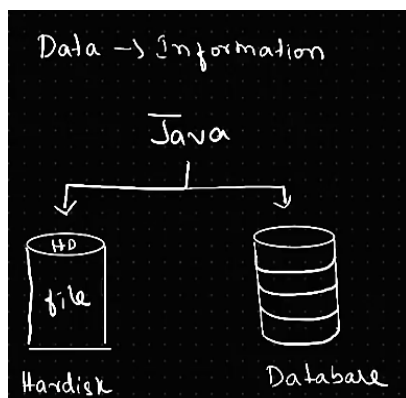
1. **Temporary Storage:** Applications and programs execute in *RAM* (Random Access Memory), which temporarily stores data. This data will be lost once the application stops or the system shuts down.
2. **Permanent Storage:** For long-term storage of data, it must be stored on a permanent medium like a hard disk or a database. This ensures data persists beyond the lifecycle of the application.

👉 File handling in Java:

- **File Storage:** If you need to store data permanently from a Java application, one option is to write the data to a file on the hard disk.

Java provides the *java.io.* package*, which includes classes and methods for IO (Input/Output) operations and file handling.

- **Persistency:** By using the *java.io.* package*, data can be persisted in a file on the hard disk, enabling it to be retrieved and reused later.



👉 Need for JDBC:

- **Database Storage:** Databases provide a more organized, secure, and scalable solution for data storage.

- **Connecting Java Applications to Databases:** JDBC enables Java programs to connect to various types of databases (local, remote, or cloud-based) and perform operations like querying, updating, and managing data.
- **Flexibility:** With JDBC, developers can interact with different databases (such as MySQL, Oracle, PostgreSQL, etc.) uniformly without worrying about the underlying database-specific details.

03: Need for a database-specific jar in the JDBC app

When developing a JDBC (Java Database Connectivity) application, it's essential to use a [database-specific JAR file](#).

★ This JAR file contains the necessary implementation of the JDBC interfaces required to connect and interact with a specific database.

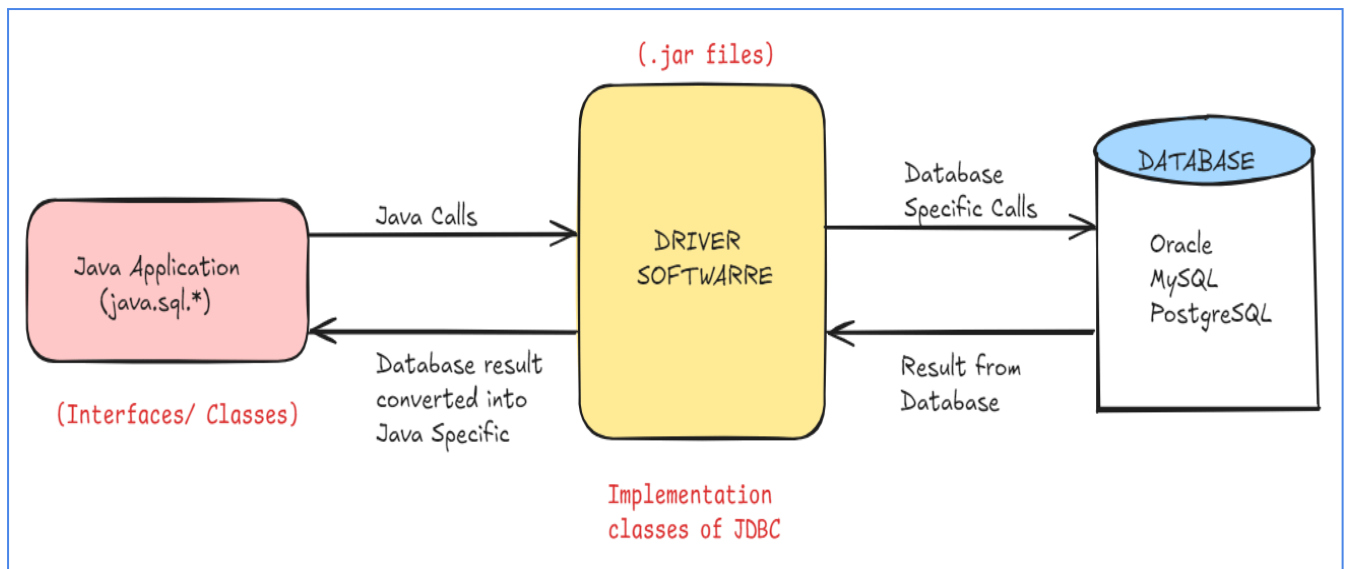
Java Provides Standard Interfaces:

- Java's JDBC API provides a set of standard interfaces that define various methods for connecting Java code with different databases. These interfaces include Connection, Statement, ResultSet, etc.
- JDBC interfaces consist of abstract methods, meaning the method names and signatures are defined, but the actual code (implementation) is not provided in the Java API.

Implementation by database vendors:

- The database vendors provide the actual implementation of the JDBC interfaces. Each database (e.g., MySQL, Oracle, PostgreSQL) has its own implementation that knows how to interact with the specific database.
- These implementations are compiled into .class files and packaged by the database vendors into a JAR file, commonly referred to as a [JDBC Driver](#).

- ➔ To use these implementations in your project, you need to download the database-specific JAR file and include it in your project's classpath.
- ➔ Once the database-specific JAR is added to your project, you can use the standard JDBC API to connect to and interact with the database.



👉 **Key Points:**

- The JDBC Driver is the implementation class of JDBC interfaces, crucial for connecting to a database as it contains the necessary code for communication.
- The standardization of JDBC interfaces enables developers to easily switch between databases with minimal code changes, provided the appropriate database-specific JAR file is included.
- While the method names and interfaces are standardized, the actual implementation is specific to each database vendor, allowing for optimized performance and compatibility with the particular database.

04: Steps Involved to Develop JDBC App (Theory)

You must correctly follow the provided steps to guarantee that your JDBC application can connect to a database, execute SQL commands, process results, and efficiently clean up resources.

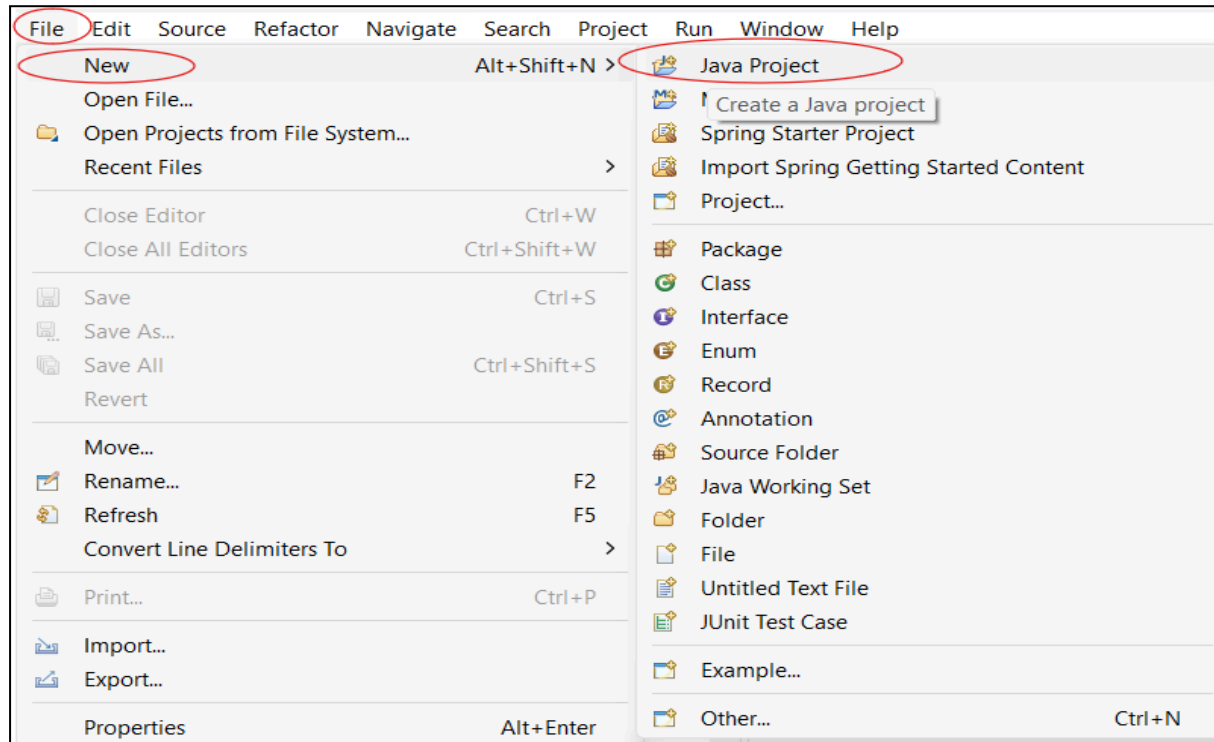
1. **Import the Required Package:** To use the JDBC API, you need to import the necessary classes and interfaces from the `java.sql` package.
2. **Load and Register the Driver:** The JDBC driver must be loaded into memory and registered with the `DriverManager` to establish a connection with the database.
3. **Establish the Connection:** Create a connection from the Java application to the database. This requires specifying the database URL, username, and password.
4. **Create the Statement:** To execute SQL queries, you need to create a `Statement` object. This object is used to send SQL commands to the database.
5. **Execute the Query:** The `Statement` object is used to execute SQL queries, such as `SELECT`, `INSERT`, `UPDATE`, or `DELETE`.
6. **Process the Result:** After executing a query, the results are processed.
7. **Close the Connection:** Close the database connection and other resources to free up resources and avoid potential memory leaks.

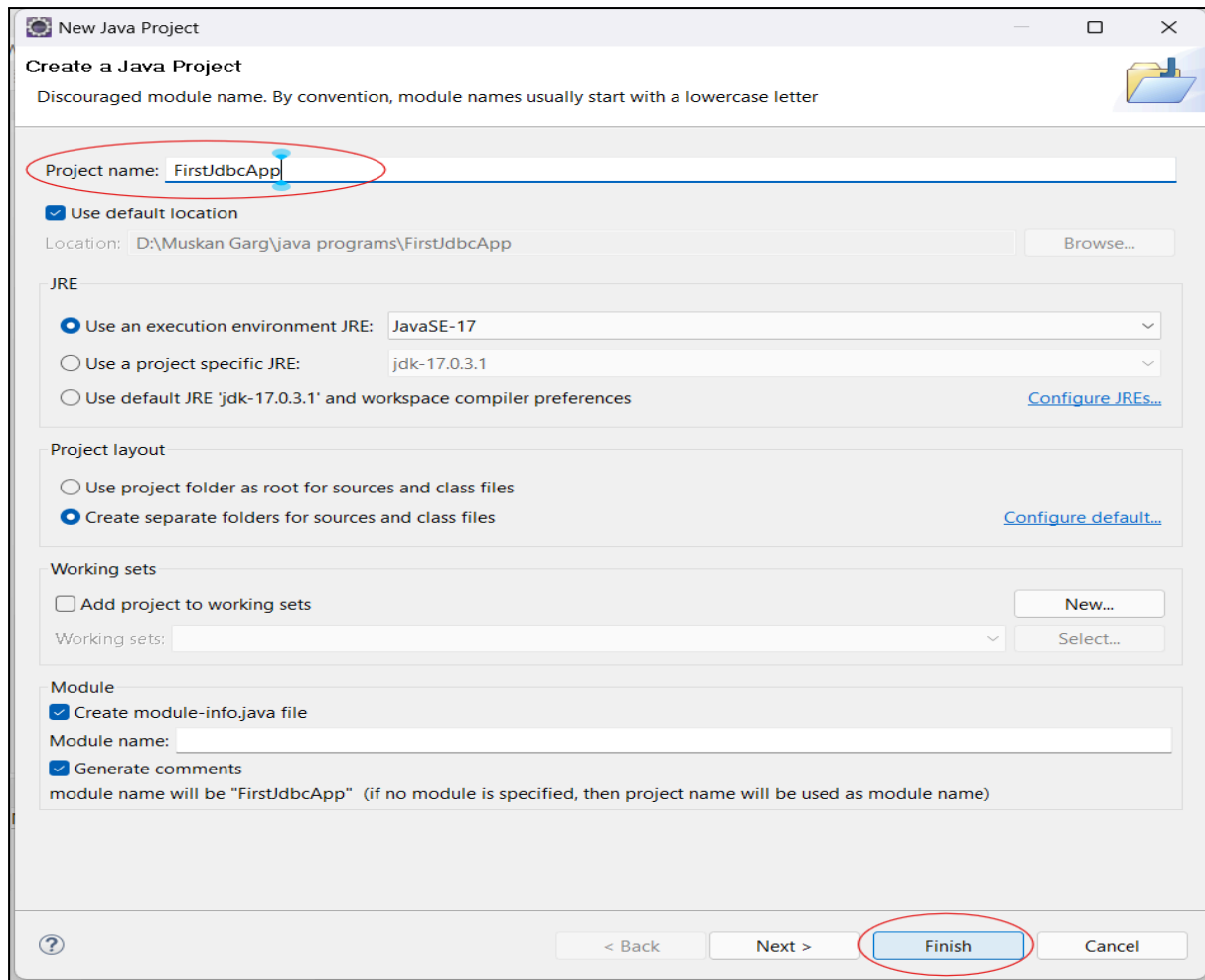
05: Setting Up JDBC Development Environment

The step-by-step guide to help you set up JDBC in your Java project:

Note: While we're using Eclipse IDE in this guide, feel free to use any IDE of your choice.

1. Create a Java Project: Start by creating a new Java project in your preferred IDE.





2. Download the JDBC Driver for MySQL:

- Visit the official MySQL connectors website to download the driver (JDBC Driver for MySQL (Connector)).

Link: <https://www.mysql.com/products/connector/>

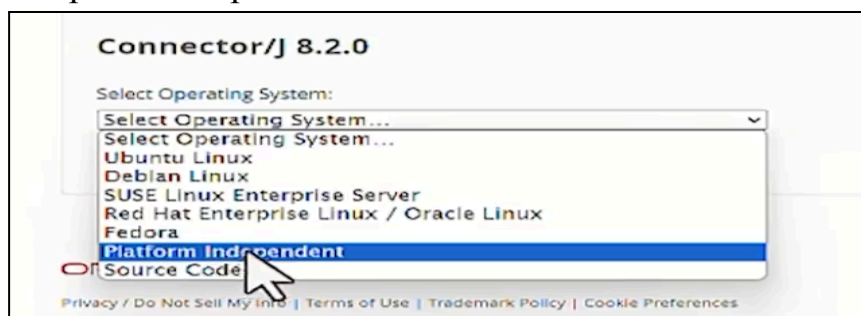
The screenshot shows the MySQL Connectors page. The main heading is "MySQL Connectors". Below it, a text block states: "MySQL provides standards-based drivers for JDBC, ODBC, and .Net enabling developers to build database C library allows developers to embed MySQL directly into their applications." A section titled "Developed by MySQL" contains a table of drivers:

Developed by MySQL	
ADO.NET Driver for MySQL (Connector/NET)	Download
ODBC Driver for MySQL (Connector/ODBC)	Download
JDBC Driver for MySQL (Connector/J)	Download
Node.js Driver for MySQL (Connector/Node.js)	Download
Python Driver for MySQL (Connector/Python)	Download
C++ Driver for MySQL (Connector/C++)	Download

The left sidebar contains a navigation menu with the following items:

- HeatWave
- MySQL Enterprise Edition
 - Datasheet (PDF)
 - Technical Specification
 - MySQL Database
 - Enterprise Backup
 - Enterprise HA
 - Enterprise Scalability
 - Enterprise Stored Programs
 - Enterprise Authentication
 - Enterprise TDE
 - Enterprise Encryption
 - Enterprise Masking

- Go to the "Select Operating System" section and choose the "Platform Independent" option.



- Download the "Platform Independent (Architecture Independent).ZIP Archive".

The screenshot shows the "Connector/J 8.2.0" download page with the "Platform Independent" option selected in the dropdown menu. Below the dropdown, there are two download options:

Download Option	Version	Size	Action
Platform Independent (Architecture Independent), Compressed TAR Archive (mysql-connector-j-8.2.0.tar.gz)	8.2.0	4.0M	Download
Platform Independent (Architecture Independent), ZIP Archive (mysql-connector-j-8.2.0.zip)	8.2.0	4.8M	Download

The "Download" button for the ZIP Archive is highlighted with a mouse cursor. MD5 hashes and a signature link are also visible for each option.

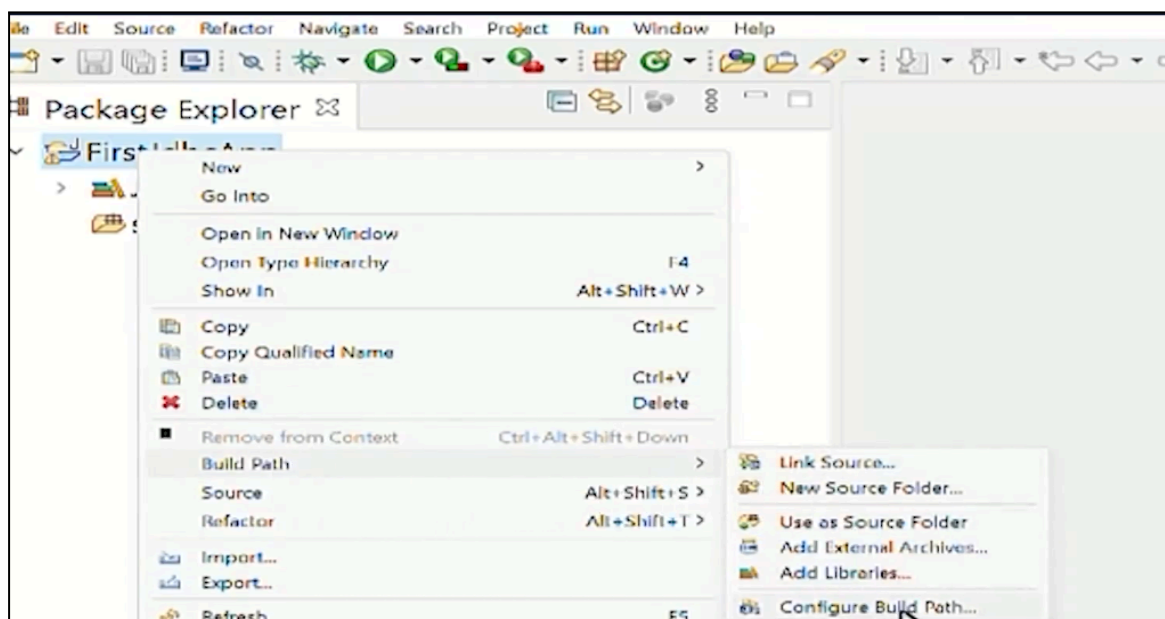
3. Extract the downloaded ZIP archive:

- Once the download is complete, extract the contents of the ZIP archive.
- After extraction, you will find the executable JAR file (e.g., mysql-connector-java-8.x.x.jar).

4. Add the JAR File to Your Java Project:

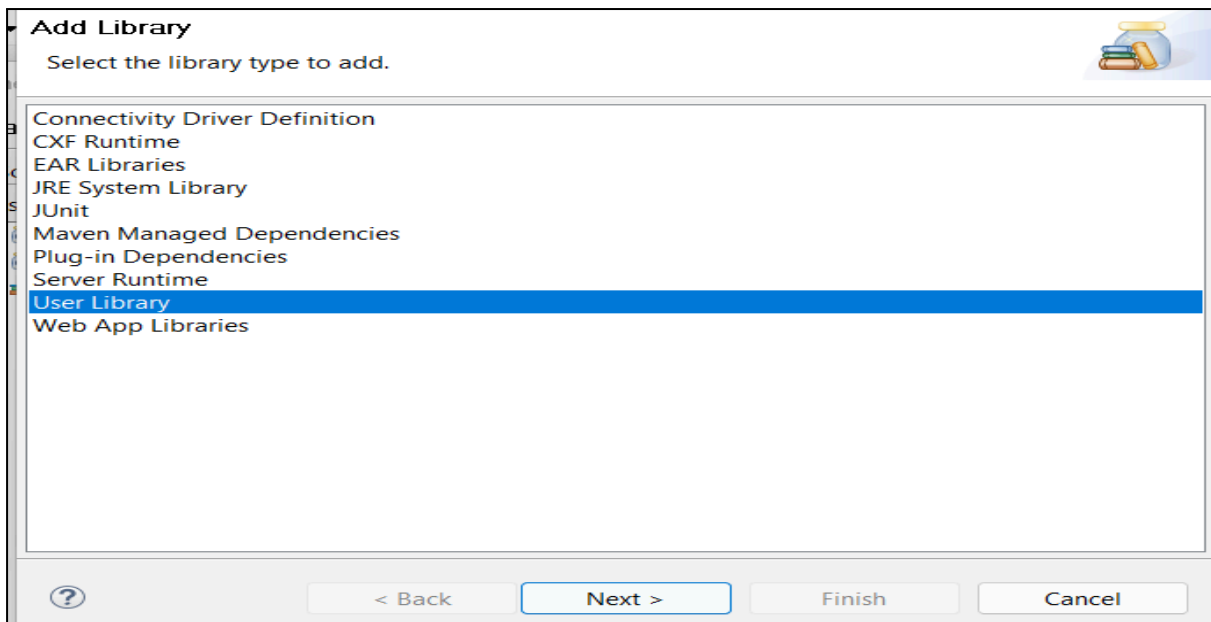
Open the Java project in which you want to add the JDBC driver.

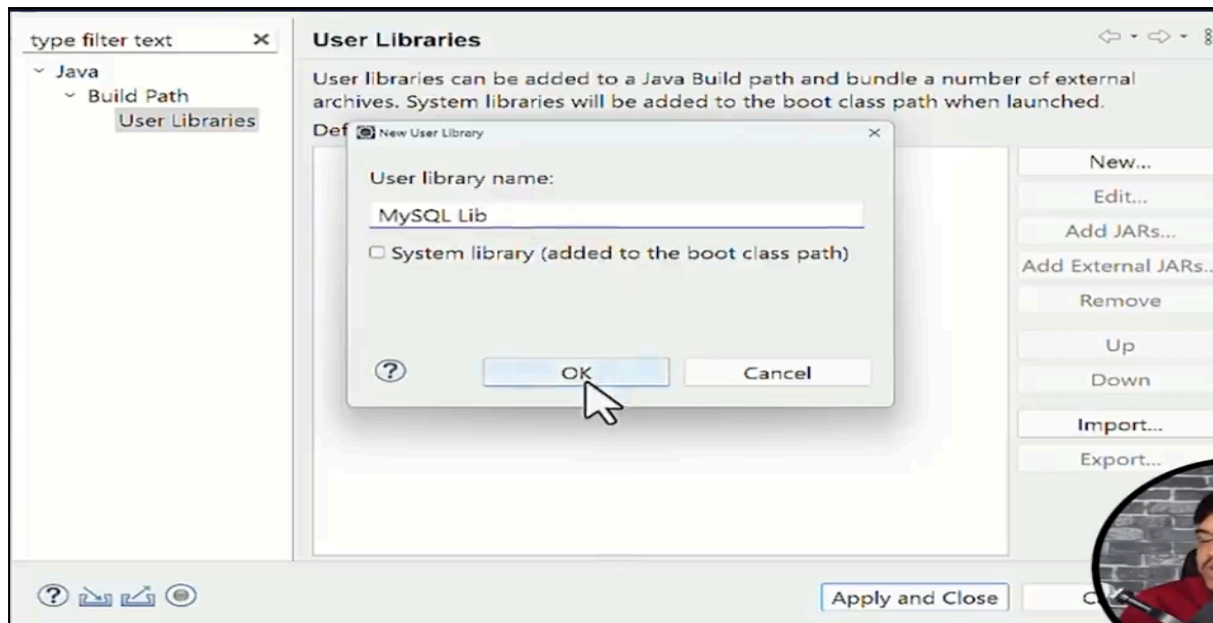
- **Configure Build Path:**
 - o Right-click on your project in the IDE.
 - o Go to **Build Path -> Configure Build Path**.
 - o A new window will open where you can configure the build path settings.



5. Create a User Library for the JDBC Driver (Optional):

- **New Library:**
 - o In the new window, you can create a new user library to keep the JAR file, making it reusable across different projects.
 - o Click on **Add Library -> User Library -> Next -> User Libraries -> New**.
 - o Give a name to the user library (e.g., MySQL Lib).

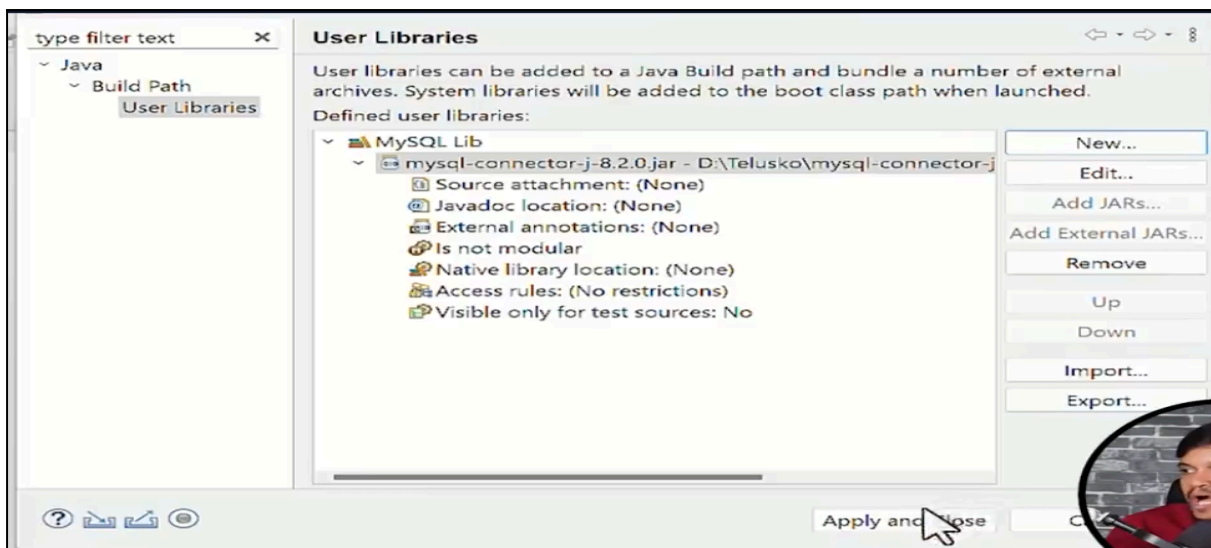
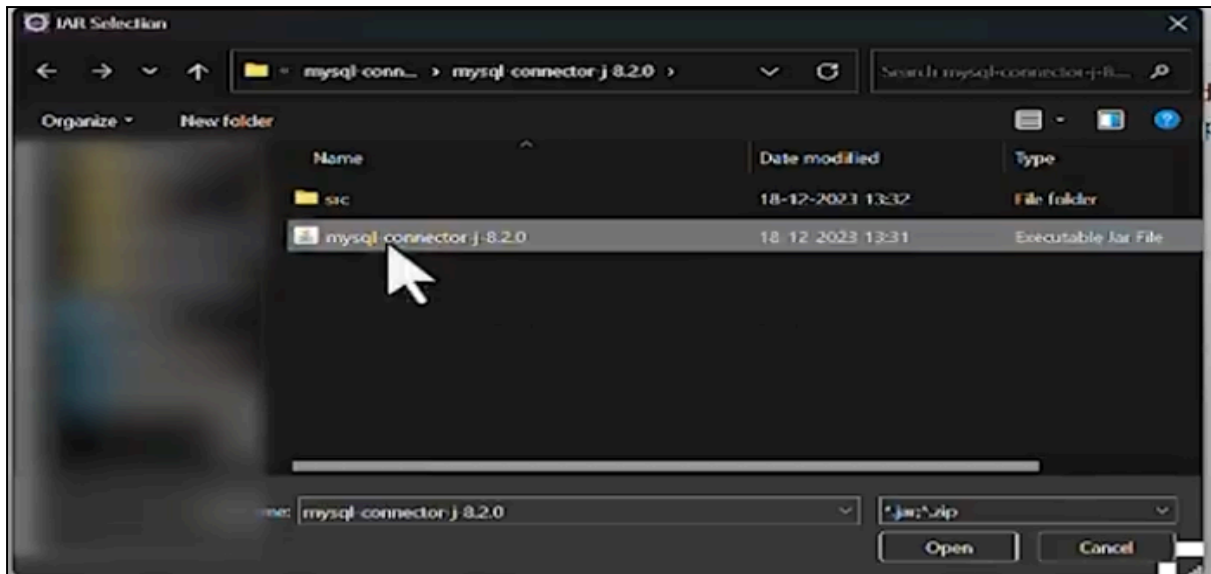




- **Add External JARs:**

- Select the newly created library.
- Click on **Add External JARs**.
- Navigate to the folder where you extracted the JAR file, select the mysql-connector-java-8.x.x.jar file, and click **Open**.
- Click **Apply and Close** to finish adding the JAR to your project.





6. Directly Add the JAR File (Alternative to User Library):

- **Add JAR:**
 - Instead of creating a user library, you can directly add the JAR file to your project.
 - Go to **Build Path -> Configure Build Path -> Libraries -> Add External JARs**.
 - Select the JAR file and apply the changes.

7. Create a New Java Class:

After setting up the JAR file, create a new Java class where you will write your JDBC code.

8. Import the JDBC Package:

In your Java class, import the necessary JDBC package to use JDBC classes and interfaces.

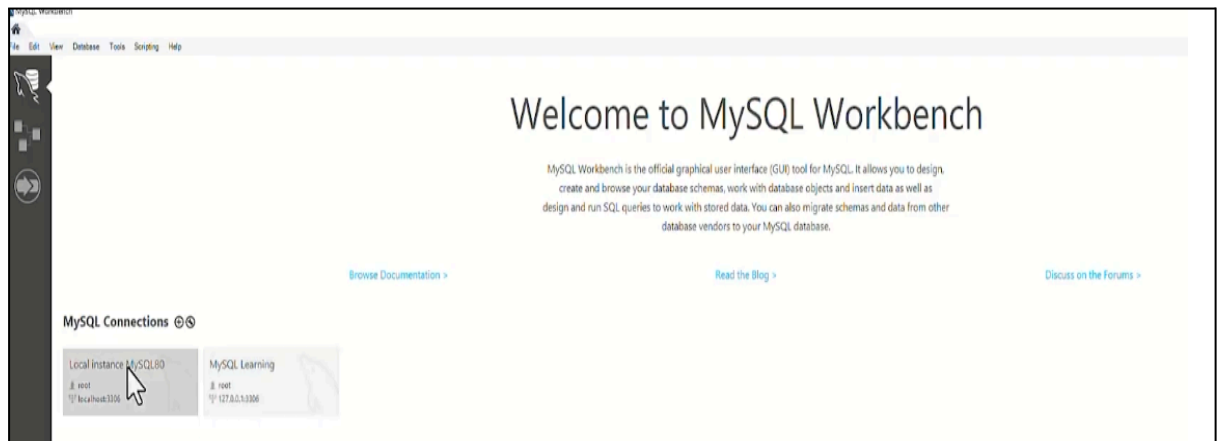
06: Setting Up Database

MySQL Workbench is a tool that can be used to create, define, and connect a MySQL database to a Java JDBC application.

Link to download MySQL Workbench :

<https://dev.mysql.com/downloads/workbench/>

1. Launch MySQL Workbench and connect to a working MySQL instance.



2. Create a new database:

```
CREATE DATABASE jdbclearning;
```

3. Use the newly created database:

```
USE jdbclearning;
```

4. Create a new table and define columns.

```
CREATE TABLE studentInfo (  
    id INT NOT NULL,  
    sname VARCHAR(40) NOT NULL,  
    sage INT NOT NULL,  
    scity VARCHAR(40) NOT NULL,  
    PRIMARY KEY(id)  
);
```

Output Table:

	id	sname	sage	scity
*				

07: Steps Involved in JDBC App (Practical)

The guide outlines the practical steps required to develop a JDBC (Java Database Connectivity) application in Java.

1. Import Required Package and Add JAR File:

- The java.sql package contains essential classes and interfaces for database operations.

```
import java.sql.*;
```

- Download the JDBC driver (e.g., MySQL Connector/J) and add the JAR file to your project's build path. This enables your Java application to communicate with the database.

2. Load and Register the Driver:

The driver converts Java calls into database-specific calls, allowing your application to interact with the database.

```
Class.forName("com.mysql.cj.jdbc.Driver");
```

- It loads the MySQL JDBC driver into memory, which registers it with the DriverManager.

3. Establish the Connection:

Create a connection to the database using the DriverManager.getConnection() method.

- **DriverManager**: This class manages the set of JDBC drivers that can be used to connect to databases. It attempts to establish a connection to the database by calling the appropriate driver.
- **getConnection()**: This method is used to establish the actual connection with the database. It returns a Connection object, which represents an active session with the database.

Parameters:

url: The full address where the database resides

user: The username for the database

password: The password for the database

```
String url = "jdbc:mysql://localhost:3306/jdbclearning";  
String user = "root";  
String password = "your_password";  
Connection connect = DriverManager.getConnection(url, user, password);
```

- In the URL string, **localhost** is used when the database resides on the same system as the application. If the database is hosted on a different system, you can replace localhost with the appropriate **IP address** of that system. The number **3306** represents the port number on which the MySQL database is listening, and **jdbclearning** is the name of the database you are connecting to.

4. Create a Statement Object:

A Statement object is needed to execute SQL queries against the database.

```
Statement statement = connect.createStatement();
```

- **connect.createStatement()** creates a Statement object that is associated with the active database connection. This Statement object allows the execution of SQL queries and commands on the database.

5. Execute the Query:

Use the Statement object to execute SQL queries, such as SELECT, INSERT, UPDATE, or DELETE.

6. Process the Result:

After executing a query, process the results returned by the database.

7. Close the Resources:

Properly closing the database connection and other resources is essential to avoid memory leaks.

```
resultSet.close();
```

```
statement.close();  
connect.close();
```

Source Code:

```
package com.telusko.jdblearning;  
import java.sql.*;  
public class LaunchApp1  
{  
    public static void main(String[] args) throws ClassNotFoundException,  
SQLException  
    {  
        // Load and Register the Driver  
        Class.forName("com.mysql.cj.jdbc.Driver");  
        // Establish the connection  
        String url = "jdbc:mysql://localhost:3306/jdbclearning";  
        String user = "root";  
        String password = "root123";  
        Connection connect = DriverManager.getConnection(url, user, password);  
        // Creating statement  
        Statement statement = connect.createStatement();  
        // Execute query  
        // Process the result  
        // Close the resources  
        statement.close();  
        connect.close();  
    }  
}
```

08: Usage of Class.forName() in JDBC

A JDBC application loads the JDBC driver class into memory using `Class.forName()`, which then automatically registers it with the `DriverManager`. This allows the application to establish a connection with the database.

- `Class.forName()` is a static method in Java that belongs to the **Class** class.
- It is used to load a class dynamically at runtime without creating an instance of the class.

★ Use of Class class:

- The `Class` class in Java is part of the `java.lang` package and is a core part of the Java Reflection API.
- The `Class` class provides a way to access metadata about classes, interfaces, arrays, and primitive types at runtime.
- Every Java class or interface that is loaded by the JVM is represented by an instance of the `Class` class.
- Using the `Class` class, you can obtain information about a class at runtime, such as its name, constructors, methods, fields, and more.

👉 Working of Class.forName():

- o **Direct Invocation:** Since `forName()` is a static method, it can be called directly using the class name.
- o **Class Loading:** When `Class.forName()` is called with a class name as a string parameter, it loads the specified class into memory. If the class has a static block, that block is executed during the class loading.

```
Class.forName("Demo");
```

- The given example loads the `Demo` class. If `Demo` contains a static block, it will be executed as part of the class loading process.

👉 To establish a connection with the database:

The given command loads the MySQL JDBC driver class (`com.mysql.cj.jdbc.Driver`) into memory, enabling the application to connect to a MySQL database.

```
Class.forName("com.mysql.cj.jdbc.Driver");
```

Alternative Method to Load the Driver:

- Instead of using `Class.forName()`, you can manually register the driver using the `DriverManager.registerDriver()` method.
- This approach explicitly registers the MySQL JDBC driver with the `DriverManager`.

```
DriverManager.registerDriver(new com.mysql.cj.jdbc.Driver());
```

★ However, starting from JDBC 4.0 and later, explicitly loading or registering the driver (either through `Class.forName()` or `registerDriver()`) is generally not necessary.

JDBC drivers are automatically loaded when an application tries to establish a connection, as long as the driver is present in the classpath. This feature, known as *automatic driver loading*, simplifies the process of working with JDBC connections.

09: Inserting Data into the Database

You can insert data into a database by executing SQL INSERT statements within your Java code.

- For non-select operations like INSERT, UPDATE, and DELETE, the `executeUpdate()` method is used.
- The `executeUpdate()` method returns an integer value indicating the number of rows affected by the SQL operation. If the value is zero, it means no rows were affected, implying the data insertion was unsuccessful.

👉 Steps for Inserting the Data:

1. Create a SQL INSERT statement:

```
String sql = "INSERT INTO studentInfo(id, sname, sage, scity) VALUES (1, 'Rohan', 17, 'Bengaluru')";
```

Here, studentInfo is the table with id, name, sage, scity as columns.

2. Execute the SQL statement:

- Use the Statement object's `executeUpdate()` method to execute the INSERT statement.
- The `executeUpdate()` method returns an integer representing the number of rows affected by the SQL operation.

```
int rowAffected = statement.executeUpdate(sql);
```

3. Check If Insertion Was Successful:

After executing the INSERT statement, check the value of the rowAffected variable to determine whether the insertion was successful.

```
if (rowAffected > 0) {  
    System.out.println("Insertion successful! Number of rows affected: " +  
        rowAffected);  
} else {  
    System.out.println("Insertion failed. No rows were affected.");  
}
```

Code:

```
import java.sql.*;

public class LaunchApp1 {

    public static void main(String[] args) {

        String url = "jdbc:mysql://localhost:3306/yourDatabaseName";

        String username = "yourUsername";

        String password = "yourPassword";


        String sql = "INSERT INTO studentInfo(id, sname, sage, scity) VALUES
(1, 'Rohan', 17, 'Bengaluru')";


        Connection conn = null;

        Statement stmt = null;


        try {

            conn = DriverManager.getConnection(url, username, password);

            stmt = conn.createStatement();

            int rowAffected = stmt.executeUpdate(sql);

            if (rowAffected > 0) {

                System.out.println("Insertion successful! Number of rows affected: "
+ rowAffected);

            } else {

                System.out.println("Insertion failed. No rows were affected.");

            }

        } catch (SQLException e) {

            e.printStackTrace();

        } finally {
```

```
try {  
    if (stmt != null) stmt.close();  
    if (conn != null) conn.close();  
} catch (SQLException e) {  
    e.printStackTrace();  
}  
}  
}
```

10 - Updating record in the database

The UPDATE SQL statement is used to modify the data in one or more rows of a table.

The `executeUpdate()` method is used for UPDATE operations, and it returns an integer indicating the number of rows affected.

Steps For Updating the Data:

1. Create an SQL UPDATE Statement:

```
String sql = "UPDATE studentInfo SET sage = 24 WHERE id = 2";
```

2. Execute the SQL Statement:

Use the Statement object's `executeUpdate()` method to execute the UPDATE statement.

```
int rowAffected = statement.executeUpdate(sql);
```

3. Check If the Update Was Successful:

After executing the UPDATE statement, check the `rowAffected` variable to determine if the update was successful.

```
if (rowAffected > 0) {  
    System.out.println("Update successful! Number of rows affected: " +  
        rowAffected);  
} else {  
    System.out.println("Update failed. No rows were affected.");  
}
```

☐ While values are hard-coded in the SQL statement in the given example, it is also possible to pass them dynamically at runtime using the `PreparedStatement`, which will be discussed later.

Code:

```
import java.sql.*;

public class LaunchApp2 {

    public static void main(String[] args) {

        String url = "jdbc:mysql://localhost:3306/yourDatabaseName";

        String username = "yourUsername";

        String password = "yourPassword";

        String sql = "UPDATE studentInfo SET sage = 24 WHERE id = 2";

        Connection conn = null;

        Statement stmt = null;

        try {

            conn = DriverManager.getConnection(url, username, password);

            stmt = conn.createStatement();

            int rowAffected = stmt.executeUpdate(sql);

            if (rowAffected > 0) {

                System.out.println("Update successful! Number of rows affected: " + rowAffected);

            } else {

                System.out.println("Update failed. No rows were affected.");

            }

        } catch (SQLException e) {

            e.printStackTrace();

        } finally {
```

```
try {  
    if (stmt != null) stmt.close();  
    if (conn != null) conn.close();  
} catch (SQLException e) {  
    e.printStackTrace();  
}  
}  
}
```

11: Retrieving the record from database

Retrieving data from a database using JDBC is a read-only operation that fetches records without altering any rows in the table. This is commonly done using the SELECT SQL statement.

- `executeQuery()` method is used to execute SELECT statements and retrieve data from the database.
- The `executeQuery()` method returns a `ResultSet` object, which holds all the records fetched from the database.

👉 Steps For Retrieving the Data:

1. Create an SQL SELECT Statement:

```
String sql = "SELECT * FROM studentInfo";
```

2. Execute the SQL Statement:

Use the Statement object's `executeQuery()` method to execute the SELECT statement.

```
ResultSet rs = statement.executeQuery(sql);
```

3. Process the ResultSet:

- The `ResultSet` object contains all the records fetched from the database. You can iterate through the `ResultSet` using a loop to access each record.
- Data from each record can be fetched using methods that start with `get` followed by the data type of the column (e.g., `getInt()`, `getString()`).
- Either the column index or name as a parameter in the method.

```
while(rs.next()){  
    //System.out.println(rs.getInt(1) + "" + rs.getString(2) + "" + rs.getInt(3)  
    + "" + rs.getString(4));  
    System.out.println(rs.getInt("id") + "" + rs.getString("sname") + "" +  
    rs.getInt("sage") + "" + rs.getString("scity"));  
}
```

4. Close the ResultSet:

After processing the data, it's essential to close the `ResultSet` and other resources.

```
rs.close();
```

Code:

```
import java.sql.*;

public class RetrieveJdbc {

    public static void main(String[] args) {

        String url = "jdbc:mysql://localhost:3306/yourDatabaseName";

        String username = "yourUsername";

        String password = "yourPassword";

        String sql = "SELECT * FROM studentInfo";

        Connection conn = null;

        Statement stmt = null;

        ResultSet rs = null;

        try {

            conn = DriverManager.getConnection(url, username, password);

            stmt = conn.createStatement();

            rs = stmt.executeQuery(sql);

            while (rs.next()) {

                System.out.println(rs.getInt("id") + " " + rs.getString("sname") + " " +
rs.getInt("sage") + " " + rs.getString("scity"));

            }

        }

    }

}
```

```
    } catch (SQLException e) {  
        e.printStackTrace();  
    } finally {  
        try {  
            if (rs != null) rs.close();  
            if (stmt != null) stmt.close();  
            if (conn != null) conn.close();  
        } catch (SQLException e) {  
            e.printStackTrace();  
        }  
    }  
}  
}
```

12 - Deleting the record from Database

The DELETE operation in JDBC is a non-select operation that affects the data in a table by removing specified records.

executeUpdate() method is used to execute DELETE SQL statements, and it returns an integer indicating the number of rows affected by the operation.

Steps for Performing a Delete Operation:

1. Create a SQL DELETE Statement:

```
String sql = "DELETE FROM studentInfo WHERE id = 2";
```

2. Execute the SQL Statement:

Use the Statement object's executeUpdate() method to execute the DELETE statement.

```
int rowsAffected = statement.executeUpdate(sql);
```

3. Check If the Delete Operation Was Successful:

After executing the DELETE statement, check the rowsAffected variable to determine if the deletion was successful.

```
if (rowsAffected == 0) {  
    System.out.println("Failed to delete the record");  
} else {  
    System.out.println("Record deleted successfully!");  
}
```

Code:

```
import java.sql.*;

public class DeleteJdbc{

    public static void main(String[] args) {

        String url = "jdbc:mysql://localhost:3306/yourDatabaseName";

        String username = "yourUsername";

        String password = "yourPassword";

        String sql = "DELETE FROM studentInfo WHERE id = 2";

        Connection conn = null;
        Statement stmt = null;

        try {

            conn = DriverManager.getConnection(url, username, password);
            stmt = conn.createStatement();
            int rowsAffected = stmt.executeUpdate(sql);
            if (rowsAffected == 0) {
                System.out.println("Failed to delete the record");
            } else {
                System.out.println("Record deleted successfully!");
            }

        } catch (SQLException e) {

            e.printStackTrace();
        }
    }
}
```

```
    } finally {  
        try {  
            if (stmt != null) stmt.close();  
            if (conn != null) conn.close();  
        } catch (SQLException e) {  
            e.printStackTrace();  
        }  
    }  
}  
}
```

13: Crud Operation with Single execute() method

- The `execute()` method in JDBC is versatile, allowing for the execution of both SELECT and non-SELECT SQL statements (such as INSERT, UPDATE, and DELETE).
- The method's return type is boolean, which indicates the type of query executed.
 - o true if a SELECT query is executed, indicating that a ResultSet is available.
 - o false if a non-SELECT query (such as INSERT, UPDATE, or DELETE) is executed, indicating that the query affected some rows.
- If the query was a SELECT, you can use the `getResultSet()` method to retrieve the ResultSet containing the query results.
- If the query was a non-SELECT, use the `getUpdateCount()` method to determine how many rows were affected by the query.

Example:

```
// String sql = "SELECT * FROM studentinfo";
String sql = "UPDATE studentinfo SET sage = 16 WHERE id = 1";

// Execute the SQL query
boolean status = statement.execute(sql);

if (status) {
    // Query was a SELECT, retrieve and process the ResultSet
    ResultSet rs = statement.getResultSet();
    while (rs.next()) {
        System.out.println(rs.getInt(1) + " " + rs.getString(2) + " " + rs.getInt(3) +
            " " + rs.getString(4));
    }
    rs.close(); // Always close the ResultSet
} else {
```

```
// Query was non-SELECT, retrieve the number of affected rows
int rows = statement.getUpdateCount();
if (rows == 0) {
    System.out.println("Operation failed!");
} else {
    System.out.println("Operation successful! Rows affected: " + rows);
}
}
```

14: Handling Exceptions and Writing better code

Exception Handling with Try-Catch Block:

- It's essential to wrap the code in a try-catch block to handle exceptions effectively.
- Multiple catch blocks can be used for a single try block, where each catch handles a specific exception.
- Include a generic catch block to handle unexpected exceptions.

Using the Finally Block:

- If an exception occurs, any code after the exception won't be executed until the catch block.
- The finally block executes after the try-catch block, regardless of whether an exception occurs.
- Use the finally block to close resources like database connections, statements, and result sets.

Declaring Common Interfaces at the Top:

- For better readability and organization, declare common interfaces like:

Connection connect = null;

Statement statement = null;

Common Steps for All Operations:

- Steps like creating connections, loading drivers, and closing resources are common across different JDBC operations.
- These steps can be encapsulated in a **utility class** to reduce redundancy and promote reusability.

Using Static Block for Driver Loading:

- Load the database drivers in a static block, so they get registered when the class is loaded.
- The static block is executed during class loading.

Creating Static Method for Connection:

- Use a static method for creating the connection, allowing direct access via the class name.

Closing Resources with a Utility Method:

- Write a separate method to close the connection and other resources.

Reusability through Utility Class:

- The utility class can be called from other classes using the class name directly, improving code maintainability.

Utility Class:

```
import java.sql.*;

public class jdbcConnection{
    static {
        // Load and Register the Driver
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
        } catch (ClassNotFoundException e) {
            e.printStackTrace();
        }
    }

    public static Connection getJdbcConnection() throws SQLException{
        //resources used in jdbc application
        Connection connection=null;
```

```
//2. Establish the connection
String url= "jdbc:mysql://localhost:3306/db_name";
String user= "root";
String password= "root";

connection=DriverManager.getConnection(url, user, password);
return connection;
}

public static void closeConnection(Statement statement, Connection
connection) throws SQLException{
    statement.close();
    connection.close();
}
}
```

Insertion Code:

```
import java.sql.*;

public class InsertionClass {

    public static void main(String[] args) {

        Connection connect = null;

        Statement statement = null;
```

```
try {  
    connect = jdbcConnection.getJdbcConnection();  
    //creating statement  
    statement = connect.createStatement();  
  
    //execute query  
    String sql = "INSERT INTO studentinfo(id, sname, sage, scity) VALUES  
(3, 'Rohit', 17, 'Pune')";  
  
    int rowAffected = statement.executeUpdate(sql);  
    //process the result  
    if (rowAffected == 0) {  
        System.out.println("Unable to insert the data");  
    } else {  
        System.out.println("Data Inserted Successfully!");  
    }  
}  
catch (SQLException e) {  
    e.printStackTrace();  
}  
catch (Exception e) {  
    e.printStackTrace();  
}
```

```
} finally {  
    //close the resources  
    try {  
        jdbcConnection.closeConnection(statement, connect);  
    } catch (SQLException e) {  
        e.printStackTrace();  
    }  
}  
  
}
```

15: Inserting Data with Prepared Statement

A **PreparedStatement** is an interface in Java that extends the Statement interface, used to execute precompiled SQL queries.

Unlike a regular Statement, where SQL queries are created and executed directly, a PreparedStatement allows you to write SQL queries with placeholders (known as parameters) and inject values dynamically at runtime.

👉 Parameterized Queries:

Prepared statements allow you to use placeholders (e.g., ?) in your SQL query, and these placeholders will be replaced with actual values at runtime.

Such queries are called parameterized queries because the data is injected into the query at runtime instead of being hardcoded in the SQL query itself.

👉 Key Features of PreparedStatement:

- **Parameterized Queries:** It allows you to define placeholders (?) within the SQL query where actual values will be substituted at runtime.
- **Improved Security:** Since the values are set using the set methods, it protects against SQL injection attacks.
- **Performance:** Prepared statements are often **precompiled**, which means that if the **same query is executed multiple times**, it can be executed faster since the query plan does not need to be recreated every time.
- **Type-Specific Methods:** It provides methods like setString(), setInt(), setDouble(), etc., for different data types, ensuring type safety when inserting data.

👉 Steps for Inserting Data with Prepared Statement:

1. **Create a PreparedStatement:** Write an SQL query with placeholders (?) for the values to be inserted. Create a PreparedStatement object with this query.

```
String query = "INSERT INTO studentinfo (id, sname, sage, scity) VALUES  
(?, ?, ?, ?)";  
PreparedStatement pstmt = connect.prepareStatement(query);
```

→ A PreparedStatement object is created by passing the query to connect.prepareStatement(query).

2. Fetching Data at Runtime: Values can be captured dynamically at runtime using input from the user using the Scanner class.

```
System.out.println("Please enter the following details to be stored in DB");  
Scanner scan = new Scanner(System.in);  
  
System.out.println("Enter your id");  
Integer id = scan.nextInt();  
  
System.out.println("Enter your name");  
String name = scan.next();  
  
System.out.println("Enter your age");  
Integer age = scan.nextInt();  
  
System.out.println("Enter your city");  
String city = scan.next();
```

3. Setting the Parameters: Prepared statements have various set methods (such as setString, setInt, setDouble, etc.) depending on the type of data you are injecting into the query.

You pass two arguments to the set method:

- The index of the placeholder in the query (1-based indexing).
- The variable name or reference to the value being inserted.

```
pstmt.setInt(1, id);
pstmt.setString(2, name);
pstmt.setInt(3, age);
pstmt.setString(4, city);
```

- 4. Execute the Query:** Call `executeUpdate()` to insert the data into the database.

```
int rowAffected = pstmt.executeUpdate();
// process the result
if (rowAffected == 0) {
    System.out.println("Unable to insert the data");
} else {
    System.out.println("Data Inserted Successfully!");
}
```

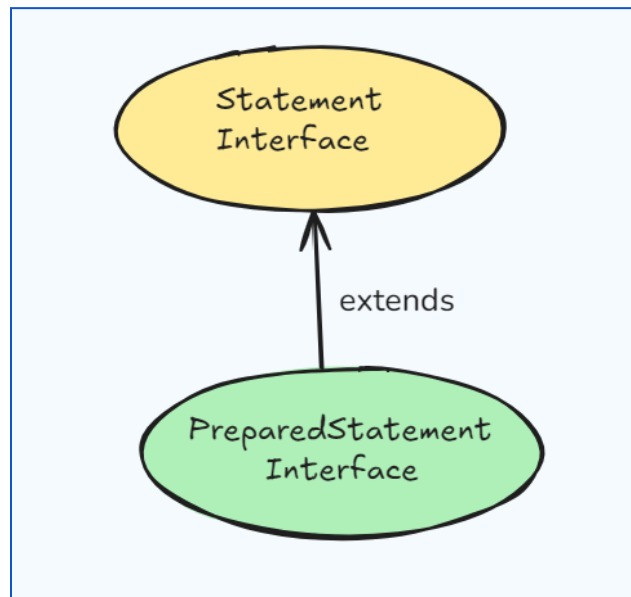
- 5. Close Resources:** Always close the `PreparedStatement` and `Connection` objects to free up resources.

```
catch (SQLException e) {
    e.printStackTrace();
} catch (Exception e) {
    e.printStackTrace();
}
finally {
    //close the resources
    try {
        JdbcUtil.closeConnection(connect, pstmt);
    } catch (SQLException e) {
        e.printStackTrace();
    }
}
```

16 - Statement VS PreparedStatement

Statement and PreparedStatement are two interfaces in Java used to execute SQL queries in a database.

- **Statement** : Parent interface for executing SQL queries.
- **PreparedStatement** : Child interface of Statement, offering more features and security.



👉 **Difference between a Statement and Prepared Statement:**

Feature	Statement	Prepared Statement
SQL Query Structure	SQL query with data is passed directly as a string.	SQL query uses placeholders (?) for parameters.
Parameterization	No parameterization; data must be embedded in the query.	Supports parameterized queries with placeholders.
Creation Method	Created using the <code>createStatement()</code> method of the connection object.	Created using the <code>prepareStatement()</code> method of the connection object.

SQL Query Requirement	No SQL query is passed when creating the Statement object. The query is passed at the time of execution.	SQL query must be passed at the time of creating the PreparedStatement object.
Security	Vulnerable to SQL injection attacks.	Safe from SQL injection due to parameterization.
Reusability	The same Statement object can be reused to execute different queries.	A new PreparedStatement object is needed for each unique SQL query.
Compilation	SQL query is compiled every time it's executed.	Query is precompiled and can be executed multiple times with different parameters, improving performance.
Efficiency	Less efficient for repeated queries, as each query is parsed and compiled separately.	More efficient for repeated execution, as the query is compiled only once.
Method for Setting Data	Data is embedded in the SQL query string.	Data is set using set methods (setString(), setInt(), etc.) for the placeholders.

Examples	
Statement	<pre>String query = "INSERT INTO users (id, name) VALUES (1, 'John')"; Statement stmt = conn.createStatement();</pre>

	<pre>stmt.executeUpdate(query);</pre>
PreparedStatement	<pre>String query = "INSERT INTO users (id, name) VALUES (?, ?)"; PreparedStatement pstmt = conn.prepareStatement(query); pstmt.setInt(1, 1); pstmt.setString(2, "Muskan"); pstmt.executeUpdate();</pre>

17: Updating the data with PreparedStatement

When updating data using PreparedStatement, we can utilize placeholders for both:

- the values we want to update
- the condition for selecting which rows to update

👉 Steps for Inserting Data with Prepared Statement:

1. **Create Parameterized Queries:** Use placeholders (?) in the SQL update query for the new values and conditions. This allows dynamic value insertion at runtime, preventing SQL injection and making the query more efficient.

```
String sql = "UPDATE studentinfo SET sage=? WHERE id=?";  
PreparedStatement pstmt = connect.prepareStatement(sql);
```

2. **User input at runtime:** Values for updating can be taken from the user at runtime.

```
Scanner scan = new Scanner(System.in);  
  
System.out.println("Please enter info that needs to be updated");  
  
System.out.println("Kindly enter your id: ");  
  
Integer id = scan.nextInt();  
  
System.out.println("Please enter your age to be updated");  
  
Integer sage = scan.nextInt();
```

3. **Setting Values:** Use the set() methods (e.g., setInt(), setString()) to bind the input values to the placeholders in the SQL query. The first parameter of the set() method is the placeholder index (starting from 1), and the second parameter is the value.

```
pstmt.setInt(1, sage);  
pstmt.setInt(2, id);
```

4. Executing the Update:

The executeUpdate() method is used to execute the update query.

```
int rowsAffected = pstmt.executeUpdate();
```

5. Checking the Result:

After executing the query, you can check if the data was successfully updated by checking the number of affected rows. If the result is greater than 0, it indicates that one or more rows were updated.

```
if(rowsAffected == 0) {  
    System.out.println("Unable to update the data");  
} else {  
    System.out.println("Data Updated Successfully!");  
}
```

Code:

```
public class UpdateStudentInfo {  
  
    public static void main(String[] args) {  
        String url = "jdbc:mysql://localhost:3306/db_name";  
        String username = "root";  
        String password = "password";
```

```

String sql = "UPDATE studentinfo SET sage=? WHERE id=?";

try (
    Connection connect = DriverManager.getConnection(url, username,
password);
    PreparedStatement pstmt = connect.prepareStatement(sql);
    Scanner scan = new Scanner(System.in);
) {
    System.out.println("Please enter info that needs to be updated");
    System.out.print("Kindly enter your ID: ");
    int id = scan.nextInt();
    System.out.print("Please enter your age to be updated: ");
    int sage = scan.nextInt();

    pstmt.setInt(1, sage);
    pstmt.setInt(2, id);

    int rowsAffected = pstmt.executeUpdate();

    if (rowsAffected == 0) {
        System.out.println("Unable to update the data. No rows affected.");
    } else {
        System.out.println("Data Updated Successfully! Rows affected: " +
rowsAffected);
    }

} catch (SQLException e) {
    e.printStackTrace();
}
}
}

```

18: Deleted the data with PreparedStatement

When deleting data using PreparedStatement, we can utilize placeholders for the value of the record that we want to delete.

Steps for Deleting Data with Prepared Statement:

1. **Create a Parameterized Query for Deletion:** Use a SQL query with placeholders (?) to delete records based on a condition (e.g., ID or name).

```
String sql = "DELETE FROM studentinfo where id=?";  
pstmt = connect.prepareStatement(sql);
```

2. **Input the User Data:** Use the Scanner class to take user input dynamically for the record that needs to be deleted.

```
Scanner scan = new Scanner(System.in);  
System.out.println("Please enter info that needs to be Deleted");  
System.out.println("Kindly enter your id: ");  
Integer id = scan.nextInt();
```

3. **Set the Value:** After receiving the input, inject the user-provided value into the query at the placeholder position using the appropriate set() method.

```
pstmt.setInt(1, id);
```

4. **Invoke executeUpdate() Method:** Call the executeUpdate() method to execute the delete query, which will return the number of rows affected.

```
int rowsAffected = pstmt.executeUpdate();
```

- 5. Check Execution:** Check the result of the executeUpdate() method to verify if any rows were deleted.

```
if (rowsAffected == 0) {  
    System.out.println("Unable to delete the data");  
} else {  
    System.out.println("Data Deleted Successfully!");  
}
```

Code:

```
public class DeleteStudentInfo {  
  
    public static void main(String[] args) {  
        String url = "jdbc:mysql://localhost:3306/db_name";  
        String username = "root";  
        String password = "password";  
  
        String sql = "DELETE FROM studentinfo WHERE id=?";  
  
        try (  
            Connection connect = DriverManager.getConnection(url, username,  
password);  
            PreparedStatement pstmt = connect.prepareStatement(sql);  
            Scanner scan = new Scanner(System.in);  
        ) {  
            System.out.print("Please enter the ID of the record to be deleted: ");  
            int id = scan.nextInt();  
  
            pstmt.setInt(1, id);
```

```
int rowsAffected = pstmt.executeUpdate();
if (rowsAffected == 0) {
    System.out.println("Unable to delete the data. No record found for the
given ID.");
} else {
    System.out.println("Data deleted successfully!");
}

} catch (SQLException e) {
    e.printStackTrace();
}
}
```

19: Retrieving the data with PreparedStatement

To retrieve data from the database using a PreparedStatement, executeQuery() method will be used.

Steps for Retrieving the Data with Prepared Statement:

1. **Select Query with Placeholders:** Create a SQL SELECT query with placeholders (?) for dynamic values.

```
String sql = "SELECT id, sname, sage, scity FROM studentinfo WHERE  
id=?";  
  
pstmt = connect.prepareStatement(sql);
```

2. **Take User Input:** Collect the necessary input from the user to determine which data to retrieve.

```
Scanner scan = new Scanner(System.in);  
  
System.out.println("Kindly enter your id for which Data to be Retrieved: ");  
  
Integer id = scan.nextInt();
```

3. **Set the User Input:** Use the appropriate set of methods to inject the user-provided input into the placeholders.

```
pstmt.setInt(1, id);
```

4. **Use executeQuery():** Execute the query using the executeQuery() method of PreparedStatement. The result of the query is stored in a ResultSet object.

```
rs = pstmt.executeQuery();
```

- 5. Process the Retrieved Data:** Check if the ResultSet contains data, and then iterate through the result set to process or display the data. Use next() method to iterate through the records.

```
if (!rs.next()) {  
    System.out.println("No data found for the given ID.");  
} else {  
    do {  
        System.out.println("ID: " + rs.getInt("id"));  
        System.out.println("Name: " + rs.getString("sname"));  
        System.out.println("Age: " + rs.getInt("sage"));  
        System.out.println("City: " + rs.getString("scity"));  
    } while (rs.next());  
}
```

Code:

```
public class RetrieveStudentInfo {  
    public static void main(String[] args) {  
        String url = "jdbc:mysql://localhost:3306/db_name";  
        String username = "root";  
        String password = "password";  
  
        String sql = "SELECT id, sname, sage, scity FROM studentinfo WHERE  
id=?";  
  
        try (
```

```

        Connection connect = DriverManager.getConnection(url, username,
password);
        PreparedStatement pstmt = connect.prepareStatement(sql);
        Scanner scan = new Scanner(System.in);
    ) {
        System.out.print("Kindly enter your ID for which data to be retrieved: ");
        int id = scan.nextInt();

        pstmt.setInt(1, id);

        ResultSet rs = pstmt.executeQuery();

        if (!rs.next()) {
            System.out.println("No data found for the given ID.");
        } else {
            do {
                System.out.println("ID: " + rs.getInt("id"));
                System.out.println("Name: " + rs.getString("sname"));
                System.out.println("Age: " + rs.getInt("sage"));
                System.out.println("City: " + rs.getString("scity"));
            } while (rs.next());
        }

    } catch (SQLException e) {
        e.printStackTrace();
    }
}

```

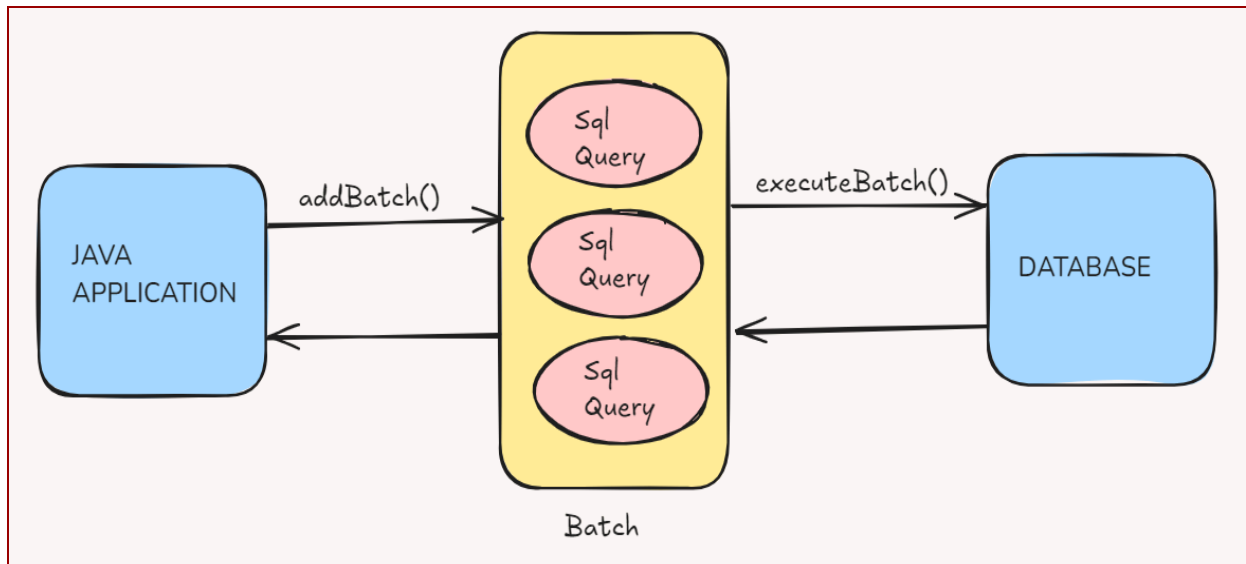
20 - Batch Update

Batch Update in JDBC allows to grouping of multiple SQL queries together and executing them as a single batch.

- It optimize database interactions by reducing the number of trips between the application and the database server.
- Instead of executing each query separately, we can add several similar or identical queries to a batch and execute them at once.
- Batch updates can be used with both Statement and PreparedStatement objects.
- It can be **applied to non-select operations** like Insertion, Updation, and Deletion.

Why Use Batch Update?

- **Efficiency:**
 - In typical JDBC operations, each SQL query is executed independently, which causes multiple database round-trips.
 - With batch updates, you reduce these round-trips, improving overall application performance, especially for large-scale operations like inserting multiple records.
- **Bulk Operations:**
 - When performing bulk operations such as mass data insertion or updating/deleting many records, batch updates streamline the process by allowing multiple similar operations to be executed simultaneously.
- **Reduced Network Traffic:**
 - Sending multiple queries in one go decreases the load on both the application and database by minimizing the number of interactions between them in the database.



👉 Methods Used in Batch Update:

- **addBatch():**

This method adds SQL commands to a batch, allowing multiple queries to be added sequentially.

- **executeBatch():**

This method executes all the queries added in the batch at once. It returns an array of integers representing the number of rows affected by each query in the batch.

👉 Steps to Perform Batch Update:

1. Batching SQL Queries:

- You add multiple SQL queries (typically of the same type) to a batch using the addBatch() method.
- This method adds the SQL query to the batch but does not execute it immediately.

```
String sql = "UPDATE studentinfo SET sage=? WHERE id=?";
```

```
pstmt = connect.prepareStatement(sql);
```

```
pstmt.setInt(1, 20);
```

```
pstmt.setInt(2, 1);
```

```
pstmt.addBatch();
```

```
pstmt.setInt(1, 24);
```

```
pstmt.setInt(2, 5);
```

```
pstmt.addBatch();
```

```
pstmt.setInt(1, 16);
```

```
pstmt.setInt(2, 44);
```

```
pstmt.addBatch();
```

2. Executing the Batch:

- Once all SQL queries are added to the batch, the `executeBatch()` method is invoked.
- This sends all queries in the batch to the database for execution at once.

```
int[] rowsAffected = pstmt.executeBatch();
```

3. Handling Results:

- After executing the batch, `executeBatch()` returns an array of integers.

- Each element in the array corresponds to the number of rows affected by each SQL query in the batch.

```
for (int i : rowsAffected) {  
    System.out.println("Rows affected: " + i);  
}
```

Code:

```
public class BatchUpdateExample {  
  
    public static void main(String[] args) {  
        String url = "jdbc:mysql://localhost:3306/studentdb";  
        String username = "root";  
        String password = "password";  
  
        String sql = "UPDATE studentinfo SET sage=? WHERE id=?";  
  
        try (  
            Connection connect = DriverManager.getConnection(url, username,  
password);  
            PreparedStatement pstmt = connect.prepareStatement(sql);  
        ) {  
            pstmt.setInt(1, 20);  
            pstmt.setInt(2, 1);  
            pstmt.addBatch();  
  
            pstmt.setInt(1, 24);  
            pstmt.setInt(2, 5);  
            pstmt.addBatch();  
  
            pstmt.setInt(1, 16);
```

```
pstmt.setInt(2, 44);  
pstmt.addBatch();  
  
int[] rowsAffected = pstmt.executeBatch();  
  
for (int i : rowsAffected) {  
    System.out.println("Rows affected: " + i);  
}  
  
} catch (SQLException e) {  
    e.printStackTrace();  
}  
}  
}
```